

Design

Student Name: David Hadzhiivanov

Student Number: 5670330

Table of Contents

Design.....	1
Table of Contents.....	1
1. Introduction	1
2. System Overview / User Journey.....	1
3. Frontend Design	2
3.1 Wireframes	2
4. Database Design	5
5. Security & Encryption Design	6
5.1 Security Implications	6
5.2 Suggested Mitigations / Options	6
6. Design Decisions & Trade-Offs.....	6
7. Conclusion.....	7

1. Introduction

This doc describes the architecture and design choices for the **Secure Notepad Web Application**. It maps user flows, backend structure, frontend views, and the security/encryption approach.

2. System Overview / User Journey

High-level user journey:

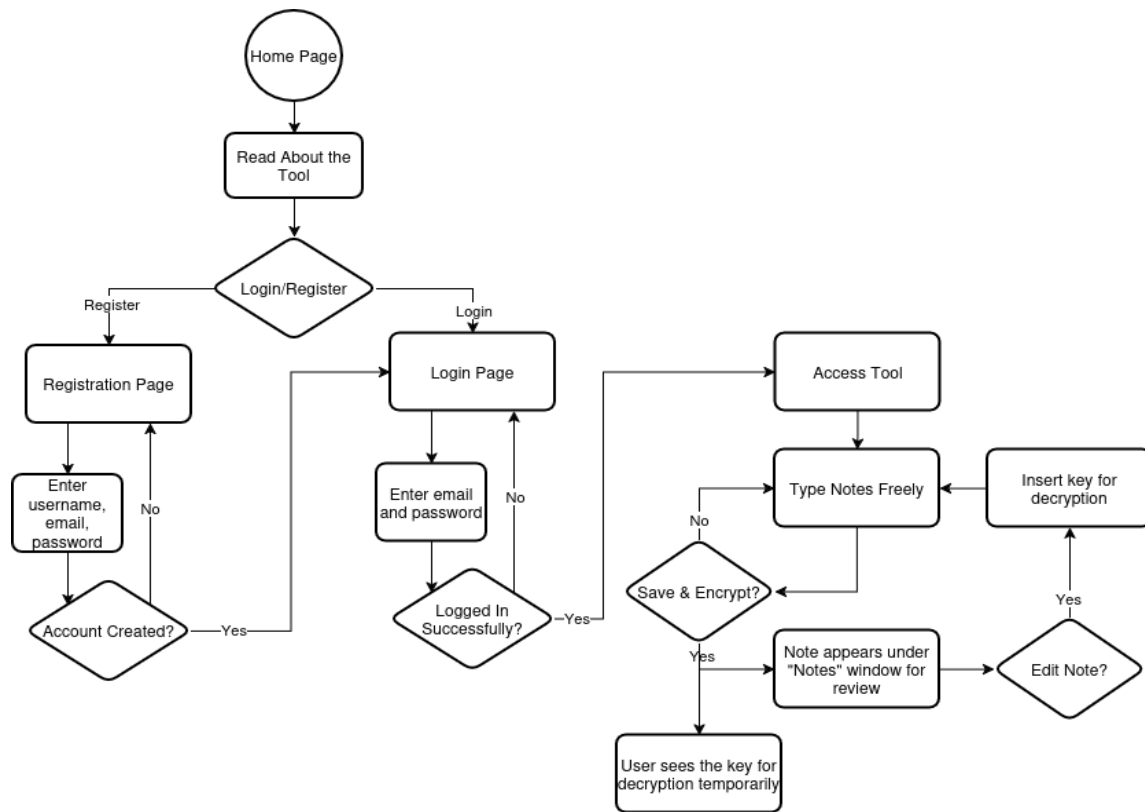


Figure 1. User journey describing the logic behind the web application

3. Frontend Design

3.1 Wireframes

Root (/) page:

NAME	user
NAME	
Text introducing the application	

Figure 2. Front page layout

Register page:

NAME	user
Register	
Username: 	
Password: 	
Register	

Figure 3. Resgistration page layout

Login page:

NAME	user
Log In	
Username: 	
Password: 	
Log in	

Figure 4. Login page layout

Notes page:

NAME	user
Notes	New
Title dd-mm-yy	Edit
Title dd-mm-yy	Edit
Title dd-mm-yy	Edit
Title dd-mm-yy	Edit

Figure 5. Notes page layout

Write notes page:

NAME	user
Write your note.	
Title: 	
Message: 	
Save	

Figure 6. Create note page layout

Decrypt / Edit notes page:

The mockup shows a web interface for editing or decrypting a note. At the top, a dark grey header bar contains the text 'NAME' on the left and 'user' on the right. Below the header, the main content area has a white background. It starts with the text 'Write your note.' in blue. This is followed by a section for editing a note, which includes a 'Key:' label and an input field, a dark grey button labeled 'Decrypt', a 'Title:' label, and a larger text area containing the placeholder text 'cyphertext'. At the bottom of this section is a dark grey button labeled 'Delete'.

Figure 7. Decrypt and edit note page layout

Delete Account Page:

The mockup shows a web interface for deleting an account. At the top, a dark grey header bar contains the text 'NAME' on the left and 'user' on the right. Below the header, the main content area has a white background. It starts with the text 'Delete Account' in blue. This is followed by a warning message: 'This will permanently remove your account and related data. You will be logged out.' Below the warning is a 'Confirm your password:' label and an input field. At the bottom of this section are two buttons: a dark grey button labeled 'Delete Account' and a blue link labeled 'Cancel'.

Figure 8. Delete account page layout

4. Database Design

User		
id	username	Password_hash

Notes			
id	User_id	title	ciphertext

Notes on key storage:

- **We do not** store user decryption keys. The design assumes keys are kept by users externally.

5. Security & Encryption Design

Encryption approach:

- Current implementation: **substitution cipher**.
 - o The decryption key is generated on the server but **never stored**.
- Operational model: User generates a key and **saves it externally**. The server only stores ciphertext.

5.1 Security Implications

Substitution cipher is **weak** cryptographically. It teaches the concept of encryption and key management, but it's **not secure** against serious attacks. This is an educational proof-of-concept only.

- If an attacker can collect enough ciphertext samples, frequency analysis can break it.
- Loss of key = irreversible loss of plaintext. No server-side recovery.

Transport: HTTPS (NGINX + certbot) is in place to protect credentials and ciphertext in transit.

5.2 Suggested Mitigations / Options

Replace substitution with AES (symmetric) for real confidentiality. AES + a secure key management strategy gives proper protection for stored notes.

6. Design Decisions & Trade-Offs

- **User-managed keys (substitution)** - choice driven by simplicity.
 - o Trade-off: weak security but simpler to implement and understand.
- **No server-side key storage** - improves confidentiality (server can't decrypt) but causes irrecoverable note loss if the user loses key. Acceptable for this project.
- **Raspberry Pi + NGINX + certbot** - gives practical hosting/SSL experience.

- Trade-off: available but not as robust as production hosting. Good for learning.
- **SQLite DB** - easy, low friction for prototype.
 - Trade-off: less concurrency and scaling.

7. Conclusion

This design document outlines the architecture, logic, and structure behind the Secure Notepad Web Application, transforming theoretical cybersecurity concepts into a practical implementation. The system integrates **Flask**, **SQLite**, and a **substitution-based** encryption method to ensure secure note handling and user authentication within a simple, efficient framework. Hosting through an **NGINX** server with automated **HTTPS** certificates via **Certbot** reinforces the project's real-world relevance. Overall, this design provides a functional foundation for a secure, scalable, and educational web application.